

- 10) Find out list of companies which are conducting placement drive using DSA
- | | |
|--------------|---------------|
| 1) Google | 6) Walmart |
| 2) Microsoft | 7) Oracle |
| 3) Amazon | 8) Salesforce |
| 4) Adobe | 9) NVIDIA |
| 5) Atlassian | 10) Uber. |

20) What is the Syllabus of DSA in Gate?

1. Basic

C programming, Recursion, Time & Space Complexity (Big O)

2. Data Structures

Arrays, Strings, Linked Lists, Stacks, Queues, Trees, Binary search trees (BST), Heaps, Hashing, Graphs

3. Searching & Sorting

Linear search, Binary search, Bubble sort, Selection sort, Insertion sort, Merge sort, Quick sort, Heap sort

4. Algorithm design Techniques

Divide and Conquer, Greedy Algorithms, Dynamic Programming, Backtracking

5. Graph Algorithms

Breadth-first Search (BFS), Depth first search (DFS).
Topological Sorting, Minimum Spanning Tree (prim's) & Kruskal's, Shortest path (Dijkstra's & Bellman ford)

Variables and datatypes in Flipkart Screen

Variable	Data type
product name	char[]
Brand	char[]
price	int
Rating	float
SellerName	char[]
product ID	int
Delivery available	bool
Address	char[]
Name	char[]
phonenumber	unsigned long int

Variables and datatypes in bus Ticket

Variable	Datatype
passenger Name	char[]
Age	int
Gender	char
Bus Number	int
Bus Name	char[]
Seat Number	int
Source	char[]
Destination	char[]
Ticket Price	float
Distance	float
Payment Status	bool

Q) Find out what are Set of variables in amazon invoice link.

Variable

Invoice Number
Order ID
Invoice data
Customer Name
Billing Address
Shipping Address
Product Name
Brand
Product ID
Quantity
Unit Price
Discount
Tax (GST)
Shipping charge
Total Amount
Payment Method
Payment Status
Delivery data

Datatype

char[]
char[]
char[]
char[]
char[]
char[]
char[]
char[]
char[]
char[]
int
float
float
float
float
float
float
float
char[]
bool
char[]

Sanctioned load

Contracted load

Previous reading

Present reading

Average units

Energy charges

Bill amount

Government Subsidy

Net Bill

Phone numbers

float

float

int
int

float int

float

float

float

float

unsigned

long int

Q) A History of tower of Hanoi

The Tower of Hanoi is a famous mathematical puzzle invented by the French mathematician Edouard Lucas in 1883.

According to a popular legend, there is a temple in India with three golden rods and 64 golden disks. Monks continuously move the disks following specific rules. It is said that when they finish moving all 64 disks, the world will end. This story was created by Lucas to make the puzzle more interesting.

Rules

- 1) Only one disk can be moved at a time.
- 2) A larger disk cannot be placed on a smaller disk.
- 3) Only the top disk of any rod can be moved.

Q) A) for ($i=1$; $i \leq n$; $i++$);
 printf ("Hello");

	Cost	Repetition	Total
$i=1$	1	$n+1$	$n+1$
$i \leq n$	1	n	n
$i++$	1	n	n
printf ("Hello")	1	n	n

$$\frac{(n+1) + n + n + n}{4} = \frac{3n+1}{4}$$

2) $i=1$;
 $i = i+20$;

printf ("%d", i);

Cost	Repetition	Total
$i = 1$	1	1
$i = i + 20$	1	1
$\text{printf}()$	1	1

3) $i = 1;$	1	1
$\text{while}(i \leq n)$	1	1
{	1	1
$\text{printf}("%d", i);$	1	1
$i++;$	1	1
}	1	1

Cost	Repetition	Total
$i = 1$	1	1
$i \leq n$	$n+1$	$n+1$
$\text{printf}(i)$	n	n
$i++$	n	n
		$3n+2$

code	Cost	Repetition	Total
echo -n "Enter the first number: "	1	1	1
read num1	1	1	1
echo -n "Enter the second number: "	1	1	1
read num2	1	1	1
Sum = \$((num1 + num2))	2	1	2
echo "the sum of \$num1 and \$num2 is : \$Sum"	1	1	1
			<u>7</u>

Hash Code

It is a number generated for every object in Java using the `hashCode()` method.

Purpose

Helps to identify objects

Makes searching and storing objects faster.

Used in Collections like `HashMap` and `HashSet`.

Program

```
class HashDemo {
```

```
    public static void main(String args[]) {
```

```
        String s = "Java";
```

```
        System.out.println("Hash Code: " + s.hashCode());
```

```
    }
```

```
}
```


HW

What are the various problems which you have observed in our surroundings. Pick any 5 problems. Find out if they can be solved using DS and Algorithms?

1) Parking slot finder - Array, Searching
Quickly finds an empty parking space in a parking area.

2) Queue Management in Hospital - Queue
Manages patients in the order they arrive, reducing confusion.

3) Lost and found item search - Hash table
Stores lost items and allows fast searching by item name or ID.

4) Student Attendance Management - Hash table, Array
Stores attendance records and quickly checks whether a student is present.

5) Bus stop waiting list - Queue
Maintains the order of passengers waiting to board the bus.

HW

Trace quick sort as the shown in the class for the following array

1) 85 76 99 13 45 103 -5 26 31

2) 350 790 400 200 100 96 900 890 760

i	j	a[i]	a[j]	a[pivot]
1) 85	76	99	13	45
a[i]				
↑				
pivot				

$a[i] \leq \text{pivot}$

$a[j] > \text{pivot}$

85 76 99 13 45 103 -5 26 31

Swap i & j

85 76 31 13 45 103 -5 26 99

Swap i & j

85 76 31 13 45 26 -5 103 99

Swap pivot & j

-5 | 76 31 13 45 26 85 103 99

Swap pivot & j

-5 | 85 31 13 45 26 76 103 99

Swap

-5 | 26 31 13 45 76 85 103 99
 i j j j j j j j
 swap i & j

-5 | 26 13 31 45 76 85 103 99
 i j i j j j j j
 j j

swap pivot & j [1] 0 [1] 0

-5 | 13 26 31 45 76 85 103 99

-5 [13] 26 31 45 76 85 | 103 99
 i j

if $i < j$ swap i & j

-5 13 26 31 45 76 85 99 103

i	j	$a[i]$	$a[j]$	$a[pivot]$
0	8	85	31	85
5	7	103	26	85
7	6	103	5	85
6	5	85	26	85
1	6	76	85	76
6	5	85	26	76
1	5	85	26	85
2	2	31	31	31
6	5	103	85	85
1	2	26	31	26
7	8	99	103	99

2) 350 790 400 200 100 96 900 890 760
 i j
 swap i & j

350 96 400 200 100 790 900 890 760
 i j
 swap i & j

350 96 100 200 400 790 900 890 760
 i j
 swap pivot & j

200 96 100 350 400 790 900 890 760
 i j
 swap j & pivot

96 200 100
 i j
 swap j & pivot

200 96 100 | 350 | 400 790 900 890 760
 i j
 swap j & pivot

100 96 200 | 350 | 400 790 900 890 760
 i j
 swap pivot & j

96 100 200 350 | 400 790 900 890 760
 i j
 swap i & j

96 100 200 350 400 | 790 900 890 760
 i j
 swap pivot & j

96 100 200 350 400 | 790 760 890 900
 i j
 swap pivot & j

96 100 200 300 400 760 790 890

i j a[i] a[j] [a[Pivot]]

~~300~~ 8 380 760 830

0 8 380 760 350

0 5 350 96 830 350

2 4 400 200 350

0 2 200 100 200

1 0 96 200 200

0 0 200 96 200

0 2 100 200 100

2 1 200 376 1400

4 8 400 760 790

5 8 790 790 790

7 6 890 900 890

7 8 890 900 890

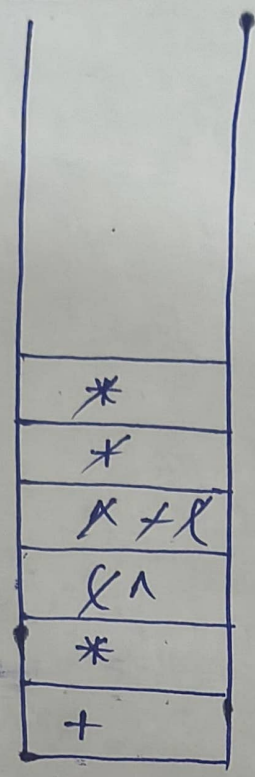
HW i char ch/c

0 a
1 +
2 b
3 *
4 (
5 c
6 ^
7 d
8 -
9 e
10)
11 ^
12 (
13 f
14 +
15 g
16 *
17 h
18)
19 -
20 i

Result

a	b	c	d	^	e	-	f	g	h	*	+	^	*	+	i	-
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Stack



while

- 1) True & 1 >= 2 = false
- 2) (→ False
- 3) ^ > - → True
- 4) * < ^ → False
- 5) (→ False
- 6) + > * → False
- 7) ^ > - → True
- * > - → True
- + = - → True

Result

^	^	0	0	*	g	A
---	---	---	---	---	---	---

Stack

→ ~~a + b * e~~ → (A * B) - (C - 0)

(	A	*	B	)	-	(	C	-	D	)
0	1	2	3	4	5	6	7	8	9	10

i char ch/c

0 (Result

1	-	A	+	*	^	1	A	B	*	/	C	D	+	-	/	^	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

2 *

3 B

Stack

4)

5 -

6 (

7 C

8 -

9 D

10)

→ (A*B) ^ (C/D)

i char ch/c

0 (

1 A

2 *

3 B

4)

5 ^

6 (

7 C

8 /

9 D

10)

Result

A	B	*	C	D	/	^
---	---	---	---	---	---	---

Stack

